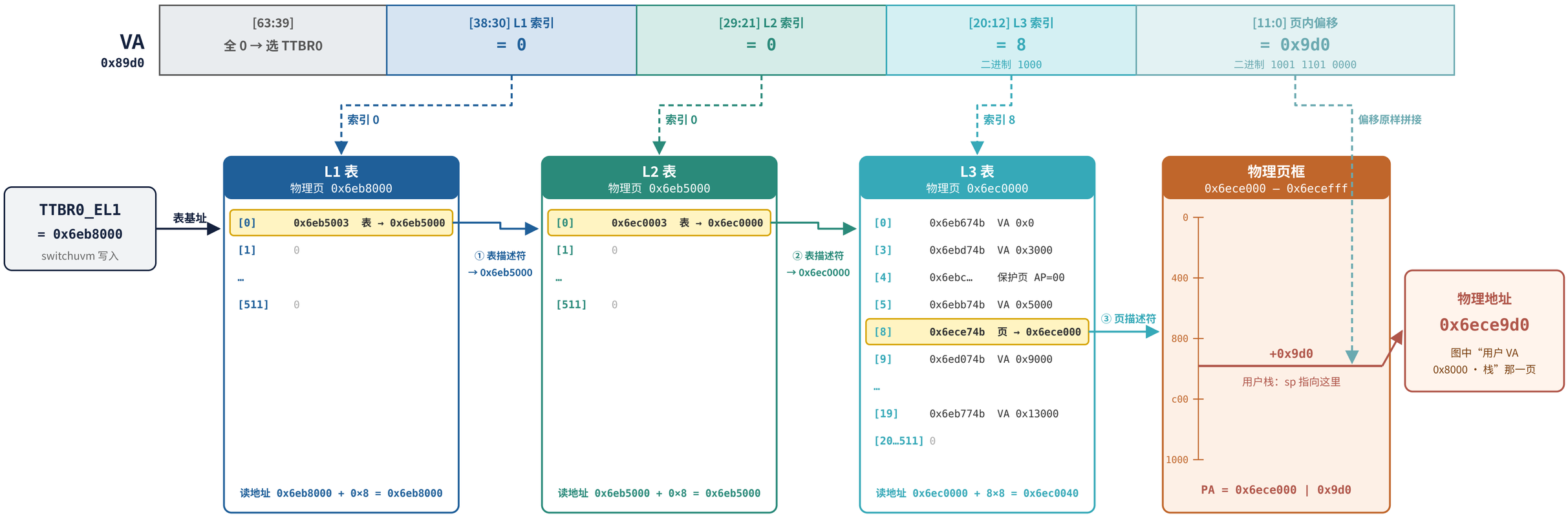
MMU 三级页表翻译实例：pid 16 的用户栈地址 VA 0x89d0 → PA 0x6ece9d0

TCR_EL1：T0SZ = 25（39 位 VA）、TG0 = 4 KB；每张表 4 KB = 512 项 × 8 B；数值取自 procvmtrace 快照



④ L3 描述符 0x6ece74b 逐位解码（同时做权限检查）

[54] UXN	[53] PXN	[47:12] 输出地址	[11] nG	[10] AF	[9:8] SH	[7:6] AP	[5] NS	[4:2] AttrIndx	[1]	[0] V
0	0	0x6ece	0	1	11	01	0	010	1	1
用户可执行	内核可执行	物理页框 0x6ece000	全局	已访问	内部共享	EL1/EL0 读写		MAIR[2] = 0xff	页	有效

AttrIndx 2 → 普通可缓存内存
（外设寄存器用 0：Device-nGnRnE）
低 12 位 0x74b = 属性；PTE_U = 第 6 位

什么时候会出错（usertrap 看到的 ESR）

某级 PTE 的 V = 0	Translation fault（DFSC 0x04–0x07，按级别）
V = 1 但 AP 不许 EL0 访问	Permission fault，例：VA 0x4000 保护页
	uvmclear 清了 PTE_U → AP=00 → DFSC 0x0F
AF = 0	Access flag fault（xv6 映射时总置 AF=1）
在 UXN=1 的页取指	Instruction abort，权限错误
VA[63:39] 既非全 0 也非全 1	直接 Translation fault，不走表

TLB：命中时跳过 ①②③

每次走表要多读 3 次内存：0x6eb8000、0x6eb5000、0x6ec0040

MMU 把结果缓存进 TLB：VA 页 0x8 → PA 页 0x6ece + 权限

下次访问 0x8xxx 直接命中，不再读页表

改页表后必须让旧项失效：

- switchuvm 写 TTBR0 后 flush_tlb()
- uvmunmap / COW 改 PTE 后刷新

xv6 不用 ASID，切换进程就整体刷新 TLB

内核地址走同一套硬件流程（TTBR1）

`struct proc  0xffffff8006ed3000`

高位全 1 → TTBR1；低 39 位 0x0006ed3000

L1 0 · L2 0x37 · L3 0xd3 · 偏移 0

→ PA 0x6ed3000（直接映射）

内核栈 `0xffffffbffffe2000`

低 39 位 0x3ffffe2000

L1 0xff · L2 0x1ff · L3 0x1e2 → PA 0x6ed2000

一句话：VA 拆成 9 + 9 + 9 + 12 位；前三个 9 位依次作三张 4 KB 表的下标，每级表项给出下一级表的物理地址，最后一级给出页框和权限；页框拼上低 12 位就是物理地址。

内核负责填表、写 TTBR、刷 TLB；查表与权限检查由 MMU 硬件完成。